MULUNGUSHI UNIVERSITY

Pursuing the frontiers of knowledge

SCHOOL OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & IT

ICT461 Advanced Web Technologies

Lab 3: Architecture Trade-Offs & Dual-Layer Validation

Item	Details
Project title	MU Library Seat Management System
Group name	Group O
Team lead (this phase)	Dada Mshisha (202200670)
Student	Dennis Sampa (202302571)
Lecturer	Kabaso B Winston (Mr.)
Due date	23rd October 2026, 23:59

Member role breakdown matrix

Role / sub-team	Assigned members	Student numbers	Primary course domain focus (Lab 3)	Comment
FE & UX	Lloyd Bulaya Joy Silishebo	202306274 202302837	Form event handling, Fetch API integration, async UI state rendering, client-side validation feedback	Loading, success and error states; field error mapping
BE & API	Cleopatra Ngoma Annette Sianongo	202301392 202302878	Backend validation middleware, JSON schema checks, RFC 7807 error responses, HTTP status codes	POST /api/lab3/checkin; 400, 409, 413, 415, 422
DB & Auth	Dennis Sampa Gift Saya	202302571 202302133	Input sanitisation, data integrity constraints, regex rules, database error mapping	validation.py; parameterised SQL; 409 conflicts
DevOps & Real-Time	Mushilo Mwewa Mayuya Dada Mshisha	202302315 202200670	API latency profiling, Postman/curl test scripts, client-bypass security verification	run_tests.sh; latency profile

1. Introduction

Lab 2 produced an accessible check-in form for the MU Library Seat Management System. Lab 3 makes that form work against a real server. The form now sends its data asynchronously with the Fetch API, and the data passes through two layers of validation: a client-side layer that gives students quick feedback, and a server-side layer that decides whether a request is accepted.

The architecture is a Flask application that renders pages on the server (Jinja2) and uses a small amount of JavaScript in the browser. The new page is served at /lab3 and posts JSON to POST /api/lab3/checkin. The validation code lives in two places: static/lab3.js in the browser and validation.py on the server. The server never trusts the browser: it re-checks every field, rejects unknown fields, sanitises free text and answers with HTTP 400, 409, 413, 415 or 422 and a structured JSON error in the RFC 7807 "problem details" format (Nottingham and Wilde, 2016).

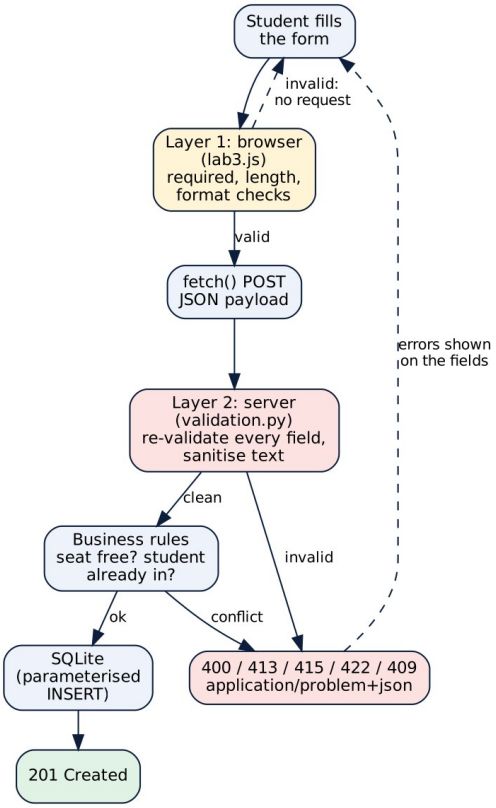


Figure 1: request flow through the two validation layers

Section 2 justifies the rendering architecture, shows the client and server code, and presents the evidence that invalid requests sent directly to the server are rejected. Section 3 explains what the lab taught.

2. Main Body

2.1 Architectural justification (Task 1)

Client-side rendering (CSR) builds the page in the browser from JavaScript and JSON. Server-side rendering (SSR) builds the HTML on the server for each request. Static site generation (SSG) builds the HTML once, at build time, and serves the same files to everyone (Osmani and Miller, n.d.). Table 1 compares them against the needs of a library seat system.

Criterion	CSR (React/Vue SPA)	SSR (Flask/Jinja2, Django, Next.js)	SSG (pre-built pages)
First load on slow campus or mobile data	Slowest: downloads and runs a JavaScript bundle before showing content	Fast: browser receives ready HTML	Fastest: files can be cached on a CDN
Live seat data	Good: fetches JSON and updates the page	Good: HTML is current on each request, JS can refresh parts	Poor: pages are frozen at build time, so seat status goes stale
Where validation is enforced	Tempting to rely on the browser	On the server, in the same process as the data	No server logic of its own, needs a separate API
Works if JavaScript fails	No: blank page	Yes: page and form still load	Yes
Build and hosting effort for a student team	Needs Node tooling, a bundler, a separate API	One Python app, no build step	Simple hosting, but cannot do check-in alone
Fit for rarely-changing pages (rules, help)	Overkill	Fine	Best

Table 1: CSR, SSR and SSG compared for the library seat system

Chosen architecture: server-side rendering with small client-side scripts. The check-in and admin pages are rendered by Flask, and JavaScript is added only where the page must react without reloading: the Fetch submission, the live validation and the seat grid, which polls every 10 seconds. The reasons are specific to this project:

- **Network and devices.** Students check in from phones inside the library. A server-rendered form arrives as usable HTML and does not wait for a framework bundle.
- **Fresh data.** Seat status changes every few seconds, so a statically generated page (SSG) would show wrong data. SSG is suitable only for the rules and help pages.
- **Trust boundary.** The rules that matter (one seat per student, 3-hour sessions, a seat cannot be double-booked) must be enforced by the server and its database. Keeping page rendering, validation and data in one Flask application means there is one place to enforce them.
- **Team size and time.** The group's code is a Flask and SQLite prototype with no build step, so a full React single-page app would add tooling and a second codebase without adding a feature the library needs.

The Lab 1 plan named React with Node.js as a possible long-term stack. That is still compatible with this work: the API contract (JSON in, application/problem+json out) and the validation rules would stay the same if the front end were later rebuilt as a CSR application. The prototype delivered in this lab is Flask SSR.

2.2 Asynchronous client-side submission (Task 2)

The form's submit event is intercepted with preventDefault(), the values are collected into a JSON object, and Fetch sends it as an HTTP POST (WHATWG, n.d.). Listing 1 shows the request. An AbortController cancels the request after 8 seconds so the page never waits forever.

Listing 1: Fetch API request (static/lab3.js)

```
  function sendCheckin(payload) {
    setLoading(true);
    var controller = new AbortController();
    var timer = window.setTimeout(function () { controller.abort(); }, 8000);   /* 8 s timeout
*/
    fetch('/api/lab3/checkin', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Accept': 'application/json,
application/problem+json' },
      body: JSON.stringify(payload),
      signal: controller.signal
    }).then(function (res) {
    ...
```

The interface is never reloaded. Table 2 lists the three states that are managed in the page, and Figures 2 and 3 show them.

State	What the user sees	What assistive technology gets
Loading	Button disabled and labelled "Checking in…"	aria-busy="true" on button and form; live region says "Sending your check-in. Please wait."
Success (201)	Green result panel with the server's JSON record; button restored	Focus moves to the result; live region: "Check-in complete. Seat G12 is yours for up to 2 hours."
Server error (400, 409, 422) or network failure	Red summary with the problem, and the error text placed on the field the server named	Summary receives focus (role="alert"); field gets aria-invalid="true" and its linked message

Table 2: user-interface states during asynchronous submission

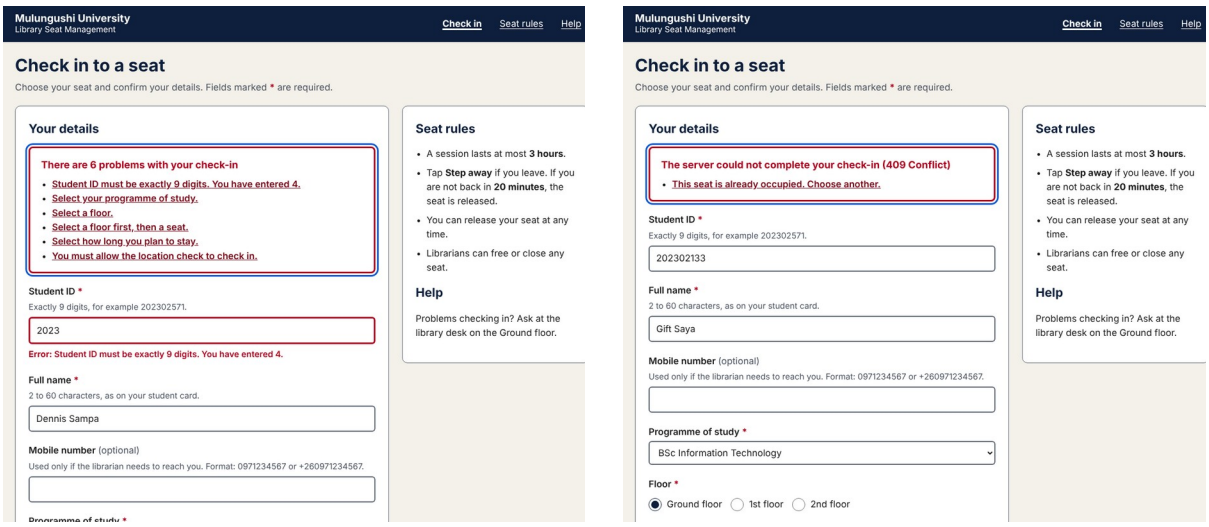


Figure 2 (left): client-side validation stops an invalid form before any request is sent. Figure 3 (right): the server answers 409 Conflict because seat G12 is already taken; the message appears in the summary and on the Seat field.

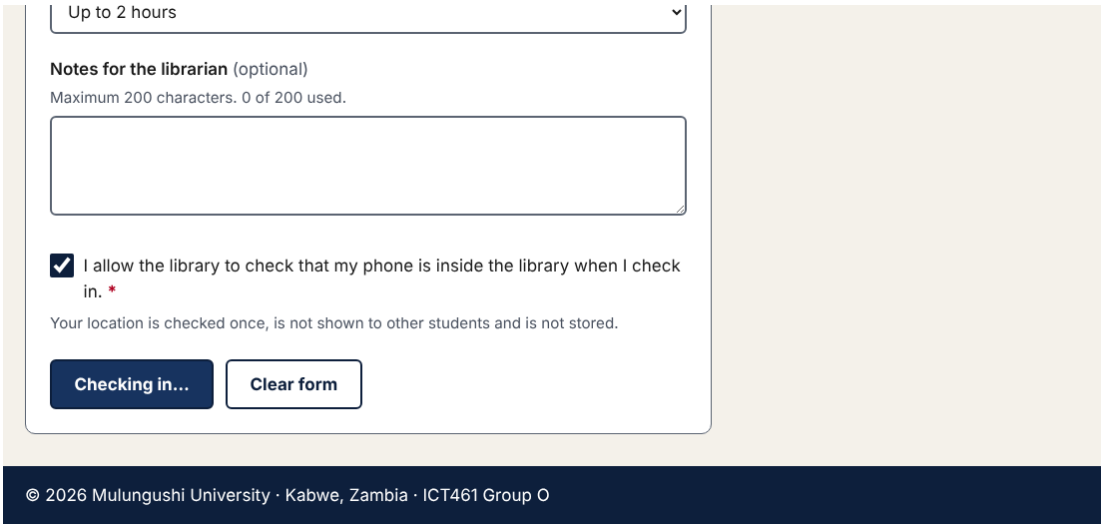


Figure 4: loading state. The response was delayed by 1.5 s in the browser test so the state could be captured.

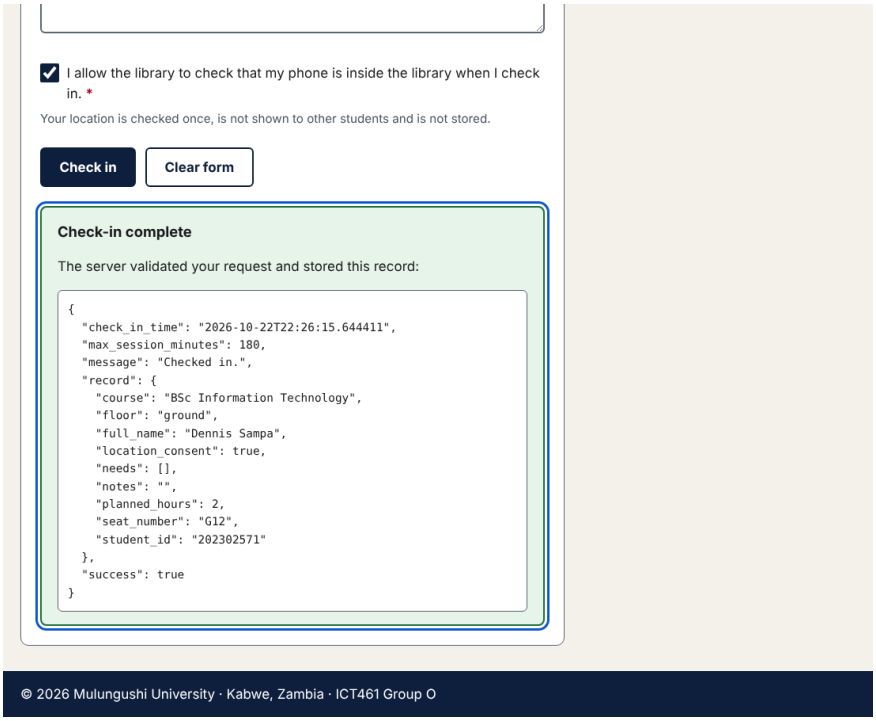


Figure 5: success state (201 Created). The panel shows the record returned by the server, with the notes field sanitised.

Browser test results (read from the page with Playwright):

```
B. During request: button disabled = true, label = "Checking in…", aria-busy = true, status =
"Sending your check-in. Please wait."
C. After success: result visible = true, label = "Check in", status = "Check-in complete. Seat G12
is yours for up to 2 hours."
   Response shown on page: { "check_in_time": "2026-10-22T22:26:15.644411", "max_session_minutes":
180, "message": "Checked in.", "record": { "course"
D. Seat already taken: summary title = "The server could not complete your check-in (409 Conflict)"
   seat aria-invalid = true, seat error = "This seat is already occupied. Choose another."
   status = "Check-in failed. Seat G12 is already occupied."
E. Network down: summary title = "Check-in not sent", message = "Could not reach the server. Check
your connection and try again.", button re-enabled
```

2.3 Client-side validation: the UX layer (Task 3)

The validation rules from Lab 2 run before any request is sent: required fields, string lengths and patterns (9-digit Student ID, name characters, Zambian mobile pattern). Fields are checked on blur and then on every keystroke. If any rule fails, the submit handler shows the error summary and returns, so no HTTP request is dispatched. The test showed 0 API requests after submitting an empty form and after entering a 4-digit Student ID (Figure 2). This layer exists for usability only; it can be switched off by the user, so it is never relied on for security.

Listing 2: client-side rules (static/lab3.js)

```
  var ID_RE = /^\d{9}$/;
  var NAME_RE = /^[A-Za-z][A-Za-z .'\-]+$/;
  var PHONE_RE = /^(\+260|0)(9|7)\d{8}$/;

  /* Rules: return an error message, or '' when valid */
  var rules = {
    'student-id': function (v) {
      if (!v) return 'Enter your student ID.';
      if (!/^\d+$/.test(v)) return 'Student ID must contain digits only.';
      if (!ID_RE.test(v)) return 'Student ID must be exactly 9 digits. You have entered ' + v.length
+ '.';
      return '';
    },
    'full-name': function (v) {
      v = v.trim();
```

```
      if (!v) return 'Enter your full name.';
      if (v.length < 2) return 'Full name must be at least 2 characters.';
      if (v.length > 60) return 'Full name must be 60 characters or fewer.';
      if (!NAME_RE.test(v)) return 'Full name can contain letters, spaces, hyphens and apostrophes
only.';
      return '';
    },
    'phone': function (v) {
      v = v.replace(/\s+/g, '');
```

2.4 Server-side validation and security enforcement (Task 4)

The endpoint treats every request as untrusted. Listing 3 shows the order of the checks, and Listing 4 the validation module. The checks are:

- **Transport:** the request must be JSON (415), under 8 KB (413) and parseable (400);
- **Fields:** every required field must be present, unknown fields are rejected (stops attempts to set fields such as is_admin), types must be correct (planned_hours must be a whole number, not the text "2"), and every value is matched against an allow-list or a strict regular expression;
- **Business rules:** a seat letter must match the floor, and a stay cannot exceed the 3-hour rule defined in database.py;
- **Sanitisation:** free text (notes, name) is normalised, stripped of control characters and HTML tags, and HTML-escaped (OWASP, n.d.a, n.d.c);
- **Conflicts:** a student who is already checked in, or a seat that is taken or out of service, gets 409 Conflict;
- **Database:** the database layer uses parameterised queries (placeholders), so values are never concatenated into SQL (OWASP, n.d.b).

Listing 3: the endpoint (app.py)

```
def api_lab3_checkin():
    if not request.is_json:
        return problem(415, "Unsupported Media Type", "Send the request with Content-Type:
application/json.")
    if request.content_length and request.content_length > 8192:
        return problem(413, "Payload Too Large", "The request body must be under 8 KB.")
    payload = request.get_json(silent=True)
    if payload is None:
        return problem(400, "Bad Request", "The request body is not valid JSON.")

    clean, errors = validate_checkin(payload)
    if errors:
        return problem(422, "Unprocessable Entity", f"{len(errors)} field(s) failed server-side
validation.", errors)

    sid, floor, seat = clean["student_id"], clean["floor"], clean["seat_number"]
    if is_already_checked_in(sid):
        return problem(409, "Conflict", "This Student ID is already checked in.", {"student_id":
"Already checked in."})
    if is_seat_out_of_service(floor, seat):
        return problem(409, "Conflict", f"Seat {seat} is out of service.", {"seat_number": "This
seat is out of service. Choose another."})
    if is_seat_taken(floor, seat):
        return problem(409, "Conflict", f"Seat {seat} is already occupied.", {"seat_number": "This
seat is already occupied. Choose another."})

    result = checkin_student(sid, clean["full_name"], clean["course"], floor, seat, None, None)
    if not result["success"]:
        return problem(409, "Conflict", result["error"])
    return jsonify({"success": True, "message": "Checked in.", "check_in_time":
result["check_in_time"],
                    ...
```

Listing 4: sanitisation and validation rules (validation.py)

```
STUDENT_ID_RE = re.compile(r"^\d{9}$")
NAME_RE = re.compile(r"^[A-Za-z][A-Za-z .'\-]{1,59}$")
PHONE_RE = re.compile(r"^(\+260|0)(9|7)\d{8}$")
TAG_RE = re.compile(r"<[^>]*>")
CONTROL_RE = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]")
```

```
def sanitise_text(value):
    """Normalise, strip control characters and HTML tags, collapse spaces, then HTML-escape."""
    value = unicodedata.normalize("NFKC", str(value))
    value = CONTROL_RE.sub("", value)
    value = TAG_RE.sub("", value)
    value = re.sub(r"[ \t]+", " ", value).strip()
    return html.escape(value, quote=True)

    sid = payload.get("student_id")
    if "student_id" not in errors:
        if not isinstance(sid, str) or not STUDENT_ID_RE.fullmatch(sid):
            errors["student_id"] = "Student ID must be exactly 9 digits."
        else:
            clean["student_id"] = sid
    ...
```

Errors follow RFC 7807: the response type is application/problem+json and includes type, title, status, detail, instance and an errors object with one message per field (Nottingham and Wilde, 2016). The client uses that errors object to place messages on the right fields (Listing 5).

Listing 5: example 422 response body

```
{
  "detail":"3 field(s) failed server-side validation.",
  "errors":{"full_name":"Full name must be 2 to 60 characters: letters, spaces, hyphens and
apostrophes only.",
  "phone":"Use a Zambian mobile number such as 0971234567 or +260971234567.",
  "student_id":"Student ID must be exactly 9 digits."},"instance":"/api/lab3/checkin",
  "status":422,"title":"Unprocessable Entity",
  "type":"about:blank"}
```

2.5 Defence-in-depth evidence

To show that the server does not depend on the browser, requests were sent with curl, with no JavaScript involved, to the running Flask server. The full terminal log is included in the code and evidence package (evidence/curl_log.txt) and is produced by the script run_tests.sh, which also serves as a repeatable test script. Table 3 summarises the 16 cases.

#	Request sent directly to the server	HTTP status	Result
1	Valid request	201 Created	Record stored
2	Same student checks in again	409 Conflict	Business rule
3	Different student, same seat	409 Conflict	Business rule
4	Empty object {}	422	7 fields reported as required
5	Short ID, letters in ID, bad phone, 1-letter name	422	Each field named in errors
6	XSS in full_name: <script>alert(1)</script>	422	Rejected by name pattern
7	SQL injection in student_id: 1; DROP TABLE active_sessions;--	422	Rejected; tables intact
8	SQL injection in seat_number: F3' OR 1=1--	422	Rejected by seat pattern
9	HTML in notes: <img src=x onerror=alert(1)>Quiet please & thanks	201	Stored as: Quiet please & thanks
10	Unknown field is_admin: true	422	Unknown field rejected
11	Floor first with seat G5	422	Seat does not match floor
12	planned_hours = 99	422	Exceeds 3-hour rule
13	planned_hours "2" and consent "yes" as text	422	Wrong data types
14	Malformed JSON	400	Bad Request
15	Content-Type: text/plain	415	Unsupported Media Type
16	20 KB body	413	Payload Too Large

Table 3: server-side tests sent with curl (bypassing the client)

Two extracts from the terminal log follow. The first is the XSS attempt in the name field; the second is the injection attempt in the Student ID.

```
$ curl -i -X POST http://localhost:5055/api/lab3/checkin -H 'Content-Type: application/json' -d
'{"student_id":"202302600","full_name":"<script>alert(1)</script>","course":"Other","floor":"first","seat_nu
mber":"F3","planned_hours":1,"location_consent":true}'
HTTP/1.1 422 UNPROCESSABLE ENTITY
Content-Type: application/problem+json

{"detail":"1 field(s) failed server-side validation.",
  "errors":{"full_name":"Full name must be 2 to 60 characters: letters, spaces, hyphens and apostrophes
only."},"instance":"/api/lab3/checkin",
  "status":422,"title":"Unprocessable Entity",
  "type":"about:blank"}
```

```
$ curl -i -X POST http://localhost:5055/api/lab3/checkin -H 'Content-Type: application/json' -d
'{"student_id":"1; DROP TABLE active_sessions;--","full_name":"Test
User","course":"Other","floor":"first","seat_number":"F3","planned_hours":1,"location_consent":t...
HTTP/1.1 422 UNPROCESSABLE ENTITY
Content-Type: application/problem+json

{"detail":"1 field(s) failed server-side validation.",
  "errors":{"student_id":"Student ID must be exactly 9 digits."},"instance":"/api/lab3/checkin",
  "status":422,"title":"Unprocessable Entity",
  "type":"about:blank"}
```

After all 16 requests, the database contained only the two valid records from tests 1 and 9, and all four tables still existed. A direct check from the browser console produced the same result: a fetch() call with an invalid payload (bypassing the form entirely) returned HTTP 422 with Content-Type application/problem+json and rejected student_id, full_name and planned_hours.

```
F. Console fetch bypassing client validation: HTTP 422 application/problem+json fields rejected =
full_name, planned_hours, student_id
```

Latency profile (DevOps focus). The 60 validation round trips measured in the browser, each one rejected with a 422 error, took on average 4.3 ms (median 4.1 ms, 95th percentile 5.7 ms, maximum 15.4 ms) on the local development server. Validation therefore adds no noticeable delay. These figures come from a single machine and are not a load test; a production test would need a real server and concurrent users.

```
G. Latency of 60 validation round trips (422): mean 4.3 ms, median 4.1 ms, p95 5.7 ms, max 15.4 ms
```

2.6 Limits of this work

- This endpoint stores the check-in but does not repeat the GPS geofence check of the main /api/checkin route, so the location consent box is recorded only in the request.
- There is no login, CSRF token or rate limiting yet. These would be needed before public use.
- Sanitisation reduces the harm of free text, but output must still be escaped wherever it is displayed (Jinja2 does this by default). Notes are not stored in the database in this prototype.
- The tests were run on the local development server, not a deployed one, and no Postman collection was made: curl was used instead.

3. Conclusion

This lab showed me that the rendering choice is a question about trust and data as much as speed. For a seat system that must stop double-booking and enforce a 3-hour limit, the server has to hold the rules, so a server-rendered application with small client scripts is a sensible fit. CSR would not make the system better for this domain, and SSG cannot show live seat status.

I also learned how asynchronous submission changes the interface. With Fetch the page must handle states a normal form submit handled for free: waiting, success, a server refusal and a network failure. Each of these needed a visible message and an announcement for screen reader users.

The most important lesson was defence in depth. The browser checks made the form pleasant to use, but the curl tests proved that they protect nothing: the same invalid data, including script and SQL injection strings, arrived at the server and was stopped there with correct status codes (400, 409, 413, 415, 422) and machine-readable errors. Client-side validation is for the user; server-side validation is for the system.

Declaration of AI use

In line with the course policy on responsible AI use, an AI assistant (Claude, by Anthropic) was used to help draft the code and the structure of this report. The code was run, the tests were executed against the running server, and the status codes, logs, screenshots and timings in this report come from those runs. I have reviewed the code and the report and I am responsible for this submission.

References

Fielding, R., Nottingham, M. and Reschke, J. (2022) HTTP Semantics. RFC 9110. IETF. Available at: https://www.rfc-editor.org/rfc/rfc9110

MDN Web Docs (n.d.) Fetch API. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN Web Docs (n.d.) AbortController. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/API/AbortController

Nottingham, M. and Wilde, E. (2016) Problem Details for HTTP APIs. RFC 7807. IETF. Available at: https://www.rfc-editor.org/rfc/rfc7807 (obsoleted by RFC 9457, 2023)

Osmani, A. and Miller, J. (n.d.) Rendering on the Web. web.dev. Available at: https://web.dev/articles/rendering-on-the-web

OWASP (n.d.a) Input Validation Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP (n.d.b) SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP (n.d.c) Cross Site Scripting Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Pallets (n.d.) Flask documentation. Available at: https://flask.palletsprojects.com/

WHATWG (n.d.) Fetch Standard. Available at: https://fetch.spec.whatwg.org/